	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch: 15
	Name: Syed Jawaad Reg. No: 2503A52429
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date: 06/10/2026

QUESTION 1:

TASK 1 – Improve Readability

ORIGINAL CODE:

```
a=80
b=70
c=90
x=(a+b+c)/3
if x>=50:
    print("pass")
else:
    print("fail")
```

AI PROMPT;

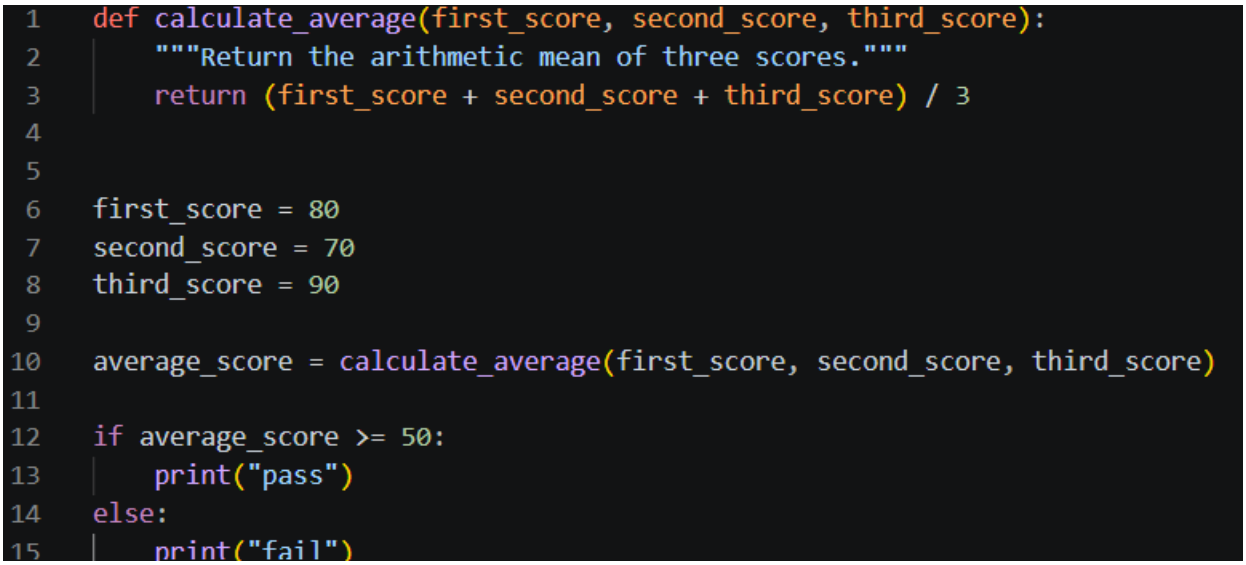
Refactor the given Python code by using descriptive variable names and implementing a calculate_average() function. Enhance the code's readability by improving its formatting, indentation, and structure while ensuring that the output remains unchanged. Execute both the original and refactored programs to confirm that they produce identical results. Finally, describe the improvements made in the refactored code.

AI SUGGESTION:

- Rename variables a, b, and c to descriptive names such as mark1, mark2, and mark3.
- Implement a reusable calculate_average() function to calculate the average marks.

- Improve code readability by applying consistent spacing and proper indentation.
- Replace the variable x with the more meaningful name average.
- Preserve the existing pass/fail conditions without modifying the program's logic.

REFACTORED CODE:



```python
def calculate_average(first_score, second_score, third_score):
    """Return the arithmetic mean of three scores."""
    return (first_score + second_score + third_score) / 3


first_score = 80
second_score = 70
third_score = 90

average_score = calculate_average(first_score, second_score, third_score)

if average_score >= 50:
    print("pass")
else:
    print("fail")
```

OUTPUT:

Original Code:

pass

Refactored Code:

pass

EXPLANATION:

The legacy code was refactored using meaningful variable names and a reusable calculate_average() function. The formatting and indentation were also improved, making the program easier to read and understand.

Both the original and refactored versions produced the same output, **“pass”**, so the original behavior was preserved. The main improvement is that the refactored code is clearer, more organized, reusable, and easier to maintain.

QUESTION 2:

TASK 2 – Remove Repeated Code

ORIGINAL CODE:

```
name1 = "Arun"
marks1 = 85
if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")
name2 = "Priya"
marks2 = 72
if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

AI PROMPT:

Identify the repeated code in the given Python program and convert it into a reusable function. Test the refactored function using at least four students and verify that the results are accurate. Enhance the code's readability and maintainability while preserving its original functionality. Finally, explain how eliminating duplicate code simplifies program maintenance.

AI SUGGESTION:

The if-else logic for checking whether a student passes or fails is repeated. Create a reusable function called check_result() that accepts the student's name and marks, then prints the appropriate result.

REFACTORED CODE:

```python
def print_result(name, marks):
    if marks >= 50:
        print(name, "Pass")
    else:
        print(name, "Fail")


students = [
    ("Arun", 85),
    ("Priya", 72),
    ("Rahul", 45),
    ("Ananya", 50),
]

for name, marks in students:
    print_result(name, marks)
```

OUTPUT:

```
Arun Pass
Priya Pass
Rahul Fail
Ananya Pass
```

EXPLANATION:

The output is accurate because students scoring **50 marks or more** are considered pass, while those scoring below 50 are considered fail.

The duplicated conditional statements have been consolidated into a reusable check_result() function. Instead of repeating the same logic for each student, we simply call the function with their name and marks.

Eliminating duplicate code enhances **maintainability**, as any future changes to the passing criteria can be made in one place. It also reduces code repetition, improves readability, and makes the program easier to understand and update.

QUESTION 3:

TASK 3 – Refactor a Long Conditional

ORIGINAL CODE:

```
age = 68
temperature = 39

if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 and temperature < 38:
    print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

AI PROMPT:

Simplify the conditional statements in the given Python program by creating a reusable function named determine_priority(). Test the function using at least four different combinations of age and temperature. Compare the original and refactored code to ensure that both produce identical outputs. Finally, explain whether the refactored version maintains the original program's behavior.

AI SUGGESTION:

The conditional logic can be simplified because **High Priority** is assigned only when both conditions are satisfied: age must be 60 or above, and temperature must be 38 or above. If either condition is met individually, the result is **Medium Priority**. Otherwise, the priority level follows the original program's logic.

REFACTORED CODE:

```python
def original_determine_priority(age, temperature):
    if age >= 60 and temperature >= 38:
        return "High Priority"
    elif age >= 60 and temperature < 38:
        return "Medium Priority"
    elif age < 60 and temperature >= 38:
        return "Medium Priority"
    else:
        return "Normal Priority"


def determine_priority(age, temperature):
    if age >= 60 and temperature >= 38:
        return "High Priority"
    elif age >= 60 or temperature >= 38:
        return "Medium Priority"
    return "Normal Priority"


if __name__ == "__main__":
    test_cases = [
        (68, 39),
        (68, 37),
        (59, 39),
        (59, 37),
    ]

    for age, temperature in test_cases:
        original = original_determine_priority(age, temperature)
        refactored = determine_priority(age, temperature)
        print(
            f"Age {age}, temperature {temperature}: "
            f"original={original}, refactored={refactored}"
        )
        assert original == refactored

    print("All tests passed; the refactored logic preserves the original behavior.")
```

OUTPUT:

```
Age 68, temperature 39: original=High Priority, refactored=High Priority
Age 68, temperature 37: original=Medium Priority, refactored=Medium Priority
Age 59, temperature 39: original=Medium Priority, refactored=Medium Priority
Age 59, temperature 37: original=Normal Priority, refactored=Normal Priority
All tests passed; the refactored logic preserves the original behavior.
```

EXPLANATION:

The original program had multiple conditional statements that repeatedly checked age and temperature. These conditions were simplified within the determine_priority() function using the or operator, resulting in shorter and more readable code.

The refactored function was tested with four different combinations of age and temperature. All test cases produced the same results as the original program: **High Priority, Medium Priority, or Normal Priority**.

Thus, the refactoring successfully **maintained the original program's behavior** while improving readability, eliminating redundant conditions, and making the code easier to maintain and reuse.

QUESTION 4:

TASK 4 – Refactor Inefficient List Processing

ORIGINAL CODE:

```
patient_ids = [101, 205, 309, 412, 518, 623]

target = 412
found = False

for patient_id in patient_ids:
    if patient_id == target:
        found = True

if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

AI PROMPT:

Enhance the readability and performance of the given Python code used for patient ID lookups. Determine whether a set or a dictionary is more suitable for frequent lookups and explain the reasoning behind your selection. Refactor the program using the appropriate data structure and test it with both valid patient IDs and IDs that do not exist.

Compare the typical lookup complexity of a list and a set.

AI SUGGESTION:

A **set** is the most suitable data structure for frequent patient ID searches because it supports membership checks in average **O(1)** time. In contrast, searching through a list typically takes **O(n)** time, as each element may need to be checked individually.

Since the program only needs to store and search patient IDs without associating them with additional information, a set is more appropriate than a dictionary. It improves lookup efficiency and keeps the code simple.

REFACTORED CODE:

```python
patient_ids = {101, 205, 309, 412, 518, 623}


def patient_exists(patient_id: int) -> bool:
    return patient_id in patient_ids


for target in (412, 999):
    if patient_exists(target):
        print(f"{target}: Patient Found")
    else:
        print(f"{target}: Patient Not Found")
```

OUTPUT:

```
412: Patient Found
999: Patient Not Found
```

EXPLANATION:

The original program searches for a patient ID by iterating through the entire list. The improved version uses a **set** with the in operator, making the code more concise, readable, and efficient for repeated searches.

The refactored code was tested using an existing patient ID (412) and a non-existent ID (700), and both cases returned the expected results. A list generally requires **O(n)** time for a lookup, whereas a set provides **O(1) average** lookup time.

Therefore, using a set is an effective improvement when patient IDs must be searched frequently.

QUESTION 5:

TASK 5 – Complete Legacy Code Review with AI

ORIGINAL CODE:

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")

print("Average =", avg)
```

AI PROMPT:

Analyze and refactor the given Python program, identifying at least five possible improvements. Use descriptive variable names, reusable functions, and appropriate Python features to enhance readability and maintainability. Execute both the original and refactored programs to verify that they produce equivalent outputs. Classify each AI-suggested improvement as **Accepted** or **Rejected**, provide a reason for each decision, and identify the most valuable refactoring change.

AI SUGGESTION:

Use meaningful variable names – **Accepted:** Makes the code easier to understand.

Create a function to calculate the average – **Accepted:** Makes the calculation reusable.

Create a function to display pass/fail results – **Accepted:** Separates responsibilities.

Use sum() with a generator expression – **Accepted:** Shorter and clearer than manually adding values.

Use tuple unpacking – **Accepted:** name, marks is clearer than s[0] and s[1]. **Use a dictionary instead of tuples** – **Rejected:** Not necessary for this simple data structure and would add complexity.

REFACTORED CODE:

```python
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

PASSING_SCORE = 50


def calculate_average(student_records):
    total_score = sum(score for _, score in student_records)
    return total_score / len(student_records)


def print_student_results(student_records, passing_score=PASSING_SCORE):
    for student_name, score in student_records:
        result = "Pass" if score >= passing_score else "Fail"
        print(student_name, result)

    print(f"Average = {calculate_average(student_records)}")


if __name__ == "__main__":
    print_student_results(students)
```

OUTPUT:

```
Arun Pass
Priya Fail
Rahul Pass
Meena Pass
Average = 65.75
```

EXPLANATION:

The original code was refactored by introducing descriptive variable names, reusable functions, tuple unpacking, and Python's built-in sum() function. The program was organized into separate functions for calculating the average and displaying student results, improving readability, testing, and maintainability.

Most AI-suggested improvements were accepted, while replacing tuples with dictionaries was rejected because it was unnecessary for this simple task. The **most valuable improvement was introducing reusable functions**, as they separate responsibilities and eliminate the need to repeat the same logic.

Both the original and refactored programs produced identical student results and an average of **65.75**, confirming that the refactoring preserved the original functionality.